

MÓDULO

Ingeniería y Analítica de Grandes Cantidades de Datos

Proyecto Big Data

Grupo 5: Uso de Bicicletas (Londres)

INTEGRANTES

Nahin Jussephe Cevallos Vines Annabella Noelia Sanchez Guzman

Martha Maritza Duran Navarrete Jesus David Suarez Aspiazu

Isabella Ivonne Martin Pimentel Angelo Saul Zurita Guerrero

Juan Andres Munizaga Torres

18 de septiembre de 2026

Código fuente: <https://github.com/Jmuniz27/proyecto-bigdata-g5-londres-bicis>

Índice

1. Resumen ejecutivo	2
2. Dataset y versiones	2
3. Metodología	3
3.1. Las tres preguntas	3
3.2. Las tres herramientas	3
3.3. Protocolo de ejecución independiente	3
3.4. Validación cruzada	4
4. Equipos de cómputo	4
4.1. Versiones de software	4
5. Fase 2: Línea base	5
5.1. Tiempo total por herramienta y versión	5
5.2. Por qué ese orden	5
5.3. Dónde se va el tiempo	5
6. Fase 3: Optimización	7
6.1. Diagnóstico	7
6.2. La optimización elegida	7
6.3. Impacto medido	7
6.4. Amortización del ETL	8
6.5. Nota sobre SQLite en la versión XL de Pc_Nahin	8
6.6. Por qué SQLite gana menos que las demás	8
7. Comparación entre equipos	9
7.1. Dónde se rompe cada máquina	9
7.2. Las tres lecturas	9
7.3. Escalado	10
8. Respuestas a las tres preguntas	10
8.1. Q1: Demanda por hora del día y día de la semana	10
8.2. Q2: Estaciones con mayor desequilibrio	11
8.3. Q3: Concentración del tráfico en rutas	12
9. Diseño del pipeline ETL	14
9.1. Herramientas y servicios por etapa	14
10. Análisis de costos	15
10.1. Costo del pipeline en la nube (mensual, estado estable)	16
10.2. Costo ya medido: cómputo local y tiempo de persona	16
11. Problemas encontrados	17
12. Limitaciones metodológicas	19
13. Conclusiones	19

1. Resumen ejecutivo

Tres motores de datos con arquitecturas distintas responden las mismas tres preguntas analíticas sobre 38.2 millones de viajes del sistema público de bicicletas de Londres, medidos en tres computadoras y en dos etapas: leyendo el CSV original y leyendo un artefacto pre-procesado.

Se compararon **Pandas** (DataFrame en memoria, columnar sobre NumPy, mono-hilo), **DuckDB** (motor OLAP columnar embebido, vectorizado y multi-hilo) y **SQLite** (motor OLTP embebido, orientado a filas). Cada combinación de versión, herramienta y pregunta se ejecutó de forma independiente, cargando el archivo desde cero, con un presupuesto de 10 minutos por ejecución.

Tres hallazgos sostienen el trabajo:

1. **El umbral de Big Data no es una cifra de filas ni de gigabytes.** La versión XL resultó imposible en una máquina de 15.7 GB de RAM y perfectamente manejable en una de 8 GB. Big Data es una propiedad de la relación entre datos, representación y hardware.
2. **Cambiar la representación mueve ese umbral sin tocar el hardware.** El equipo que falló leyendo el CSV de 4.7 GB completó la misma carga de trabajo en 6.0 segundos sobre el artefacto optimizado.
3. **La arquitectura de almacenamiento pesa más que las especificaciones.** DuckDB resolvió el dataset completo en 2.7 segundos en la máquina de menor RAM del grupo, mientras SQLite necesitaba varios minutos en la de 40 GB. Un índice B-tree no convierte un motor OLTP en columnar.

2. Dataset y versiones

La fuente es el conjunto [Bikeshare usage in London and Taipei](#) de Kaggle. **El alcance de este trabajo es únicamente la red de Londres**; los datos de Taipéi quedan fuera por decisión de equipo.

Se trabajó con el histórico de viajes en cinco versiones de tamaño creciente, más `london_stations.csv`, un catálogo de 802 estaciones con nombre, latitud y longitud.

Cuadro 1: Versiones disponibles del dataset

Versión	Archivo	Tamaño (GB)	Filas	Factor vs. XS
XS	<code>london_xs.csv</code>	0.049	398,075	1.0×
S	<code>london_s.csv</code>	0.488	3,972,436	10.0×
M	<code>london_m.csv</code>	1.181	9,590,698	24.1×
L	<code>london_l.csv</code>	2.363	19,148,881	48.1×
XL	<code>london.csv</code>	4.725	38,215,560	96.0×

El esquema tiene nueve columnas: `rental_id`, `duration`, `bike_id`, `start_rental_date_time`, `end_rental_date_time`, `start_station_id`, `start_station_name`, `end_station_id`, `end_station_name`.

La exploración inicial, hecha con DuckDB en streaming para no cargar los archivos grandes en memoria, detectó tres problemas de calidad:

- `end_station_id` tiene nulos, que corresponden a viajes sin devolución registrada. Se excluyen de las llegadas y de los pares origen-destino.
- `duration` contiene ceros y valores extremos de varios días. No intervienen en las tres preguntas, pero se documentan.

- `start_station_id` es entero mientras `end_station_id` se infiere como `DOUBLE` por causa de los nulos. Hay que castear explícitamente para que los identificadores comparen igual en las tres herramientas.

El rango de fechas cambia con la versión: las versiones chicas cubren pocas semanas y las grandes cubren más meses. Eso altera la interpretación de la estacionalidad, no el método.

3. Metodología

3.1. Las tres preguntas

Cuadro 2: Preguntas analíticas del proyecto

#	Pregunta	Salida
Q1	¿Cómo varía la demanda de viajes por hora del día y día de la semana?	168 combinaciones día × hora
Q2	¿Qué estaciones tienen mayor desequilibrio entre llegadas y salidas?	808 estaciones con actividad
Q3	¿Existen rutas origen-destino que concentran gran parte del tráfico?	510,672 rutas distintas

3.2. Las tres herramientas

Cuadro 3: Herramientas comparadas

Herramienta	Arquitectura	Por qué está en la comparación
Pandas	DataFrame en memoria, columnar sobre NumPy, mono-hilo	Caso base: rápido mientras el dataset quepa en RAM
DuckDB	Motor OLAP columnar embebido, vectorizado y multi-hilo	Diseñado exactamente para este tipo de agregación
SQLite	Motor OLTP embebido, orientado a filas	Misma interfaz SQL, arquitectura de almacenamiento opuesta

SQLite es la tercera herramienta y aporta el contraste más informativo del trabajo: comparte el lenguaje con DuckDB pero no su modelo de almacenamiento, así que aísla el efecto de la arquitectura frente al del lenguaje de consulta.

3.3. Protocolo de ejecución independiente

Cada combinación de versión, herramienta y pregunta es una **ejecución independiente que arranca desde el archivo**: se carga desde cero y después se corre la consulta. El tiempo reportado es carga + query, y ese es el número que se compara contra el límite de 10 minutos.

La razón es que SQLite no conserva el archivo cargado entre consultas, a diferencia de Pandas, que mantiene el DataFrame en RAM, y de DuckDB, que mantiene la tabla en memoria. Aplicar el mismo protocolo a las tres evita que alguna se beneficie de reutilizar una carga que otra tuvo que repetir.

Otras reglas que sostienen la honestidad de la medición:

- La carga a SQLite **no usa Pandas**. Se lee el CSV en streaming con el módulo `csv` de la librería estándar y se inserta por lotes de 200,000 filas con `executemany`. Usar `df.to_sql()` mediría a Pandas disfrazado de SQLite.
- DuckDB **materializa** la tabla con `CREATE TABLE . . . AS SELECT`, en vez de leer el CSV de forma perezosa, para competir en igualdad de condiciones con `pd.read_csv()`.
- La base SQLite se crea desde cero en cada ejecución y se borra al terminar.
- El pico de RAM se captura con un hilo que muestrea el proceso cada 50 ms. Medir solo antes y después perdería el pico transitorio que ocurre dentro de la lectura.
- Los días de la semana se normalizan a lunes = 0 en las tres herramientas, porque cada motor los numera distinto.

3.4. Validación cruzada

Un benchmark solo vale si las tres herramientas responden lo mismo. Los resultados se comparan celda a celda contra Pandas como referencia, con tolerancia numérica en las columnas de porcentaje, y se verifica el cuadro aritmético: la suma de viajes de Q1 debe igualar el número de filas del archivo, y las llegadas de Q2 deben igualar el total de Q3. **Las tres herramientas produjeron resultados idénticos en todas las versiones que completaron.**

4. Equipos de cómputo

El barrido se corrió en tres máquinas con perfiles deliberadamente distintos. Esa diversidad es lo que permite responder dónde empieza a ser Big Data, porque la respuesta cambia con el equipo.

Cuadro 4: Ficha técnica de los equipos utilizados

Equipo	Sistema	Procesador	Núcleos (físicos/lógicos)	RAM	Versiones corridas
Pc_Jesus	macOS Darwin 27.0 (arm64)	Apple arm, 3.20 GHz	8 / 8	8.0 GB	M, L, XL
Pc_Martha	Windows (AMD64)	11 Intel i7 gen 13, 1.70 GHz	10 / 12	15.7 GB	XS, S, M, L, XL
Pc_Nahin	Windows (AMD64)	11 Intel Core Ultra 9 185H, 2.30 GHz	16 / 22	39.4 GB	M, L, XL

4.1. Versiones de software

Las versiones no son idénticas entre equipos. La diferencia es menor frente a las brechas de orden de magnitud que se observan, pero se declara por transparencia.

Las tres máquinas corren DuckDB 1.5.5, así que las diferencias de rendimiento entre ellas no se explican por la versión del motor analítico.

Cuadro 5: Versiones de software por equipo

Equipo	Python	pandas	duckdb	sqlite	numpy
Pc_Jesus	3.14.3	3.0.5	1.5.5	3.50.4	2.5.3
Pc_Martha	3.12.10	3.0.2	1.5.5	3.49.1	2.4.4
Pc_Nahin	3.14.0	3.0.1	1.5.5	3.50.4	2.4.3

5. Fase 2: Línea base

DuckDB es entre 7 y 20 veces más rápido que Pandas y entre 30 y 40 veces más rápido que SQLite sobre el mismo CSV. El orden se mantiene en las tres máquinas y en todas las versiones.

5.1. Tiempo total por herramienta y versión

Suma de las tres preguntas, en segundos. Cada celda es carga + query acumulado.

Cuadro 6: Tiempo total (carga + query) por versión, equipo y herramienta

Versión	Equipo	Pandas	DuckDB	SQLite
XS	Pc_Martha	1.9	0.7	5.8
S	Pc_Martha	29.5	4.0	97.2
M	Pc_Jesus	34.5	5.0	108.2
M	Pc_Martha	50.0	6.7	164.6
M	Pc_Nahin	143.1	15.3	445.0
L	Pc_Jesus	79.1	11.5	232.8
L	Pc_Martha	106.5	15.3	404.0
L	Pc_Nahin	288.1	25.9	776.9
XL	Pc_Jesus	195.4	27.3	472.6
XL	Pc_Martha	falló	falló	falló
XL	Pc_Nahin	501.4	122.3	excedió

5.2. Por qué ese orden

DuckDB almacena por columnas y ejecuta de forma vectorizada y multi-hilo. Para un GROUP BY lee únicamente las dos o tres columnas que necesita; el resto del ancho de fila nunca toca el disco ni la RAM.

Pandas también es columnar en memoria y vectorizado sobre NumPy, pero es mono-hilo y paga materializar las nueve columnas, incluidas las dos de texto con los nombres de estación, que son las más caras de parsear y las que más memoria ocupan.

SQLite está orientado a filas. Un GROUP BY recorre cada registro completo y parsea los timestamps con `strftime` fila por fila. Es el comportamiento esperado: SQLite está optimizado para OLTP, es decir, buscar e insertar pocas filas por clave, no para agregar decenas de millones.

5.3. Dónde se va el tiempo

El desglose entre carga y consulta es la observación que justifica toda la Fase 3. En la versión XL sobre Pc_Jesus:

La carga domina en todos los casos. Y como el protocolo obliga a recargar el archivo en cada pregunta, ese costo se paga tres veces. **El formato de almacenamiento es, entonces, la palanca de optimización más rentable disponible.**

Cuadro 7: Desglose carga vs. query, versión XL, Pc_Jesus

Herramienta	Pregunta	Carga (s)	Query (s)	% en carga
Pandas	Q1	56.7	2.7	95 %
Pandas	Q3	57.2	21.6	73 %
DuckDB	Q1	9.0	0.3	97 %
DuckDB	Q3	8.1	1.4	85 %
SQLite	Q1	99.8	28.6	78 %
SQLite	Q2	108.8	82.6	57 %

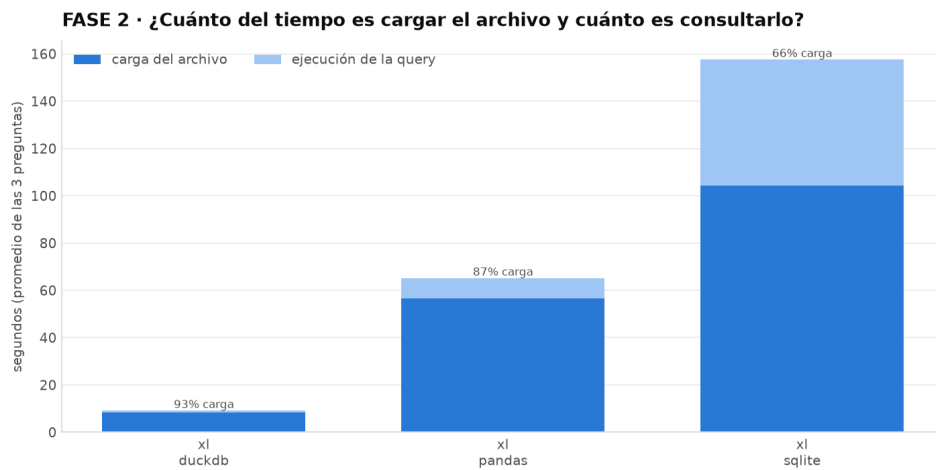


Figura 1: Reparto entre tiempo de carga y tiempo de consulta, Pc_Jesus.

FASE 2 · Escalado del tiempo con el tamaño del dataset — jesus-macbook

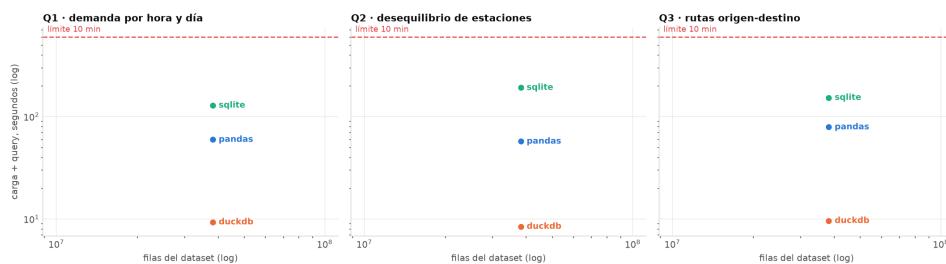


Figura 2: Escalado log-log del tiempo total contra el número de filas, Pc_Jesus.

6. Fase 3: Optimización

Un paso de ETL ejecutado una sola vez por versión reduce el tiempo de análisis entre 10× y 110×, y se amortiza antes de completar una sola corrida.

6.1. Diagnóstico

De la Fase 2 salen cuatro hechos que definen qué optimizar:

1. La carga domina el tiempo total, y como cada pregunta recarga el archivo, ese costo se paga tres veces.
2. Se leen nueve columnas y solo se usan tres: `start_rental_date_time`, `start_station_id` y `end_station_id`. Las dos columnas de nombres de estación son las más pesadas y además son redundantes, porque ya están normalizadas en el catálogo.
3. Los tipos están sobredimensionados. Los identificadores de estación llegan a ~800 y viven en INT64 o DOUBLE; la hora y el día se recalculan desde el timestamp en cada consulta.
4. SQLite recorre filas completas y parsea timestamps con `strftime` registro por registro.

6.2. La optimización elegida

Pre-procesar a un artefacto columnar con tipos reducidos, sobre el cual corren después todas las consultas.

Cuadro 8: Decisiones de la optimización

Decisión	Por qué mejora el rendimiento
Formato Parquet, para Pandas y DuckDB	Binario y columnar: el motor lee solo las columnas de la consulta y no parsea texto. Trae esquema y tipos embebidos, así que no hay inferencia.
CSV estrecho de 4 enteros, para SQLite	SQLite no lee Parquet. Su artefacto es un CSV con cuatro columnas numéricas cortas: el parseo por fila se abarata frente a nueve campos con dos cadenas largas y dos timestamps.
Proyección a 4 columnas	Se eliminan <code>rental_id</code> , <code>bike_id</code> , <code>duration</code> , los dos nombres y <code>end_rental_date_time</code> . Menos bytes leídos por fila.
Pre-cálculo de <code>dia_semana</code> y <code>hora</code>	Se hace una vez en el ETL en vez de en cada consulta, y se guardan como TINYINT.
Tipos reducidos (TINYINT, SMALLINT)	Menos memoria por fila y mejor aprovechamiento de caché.
Ordenar por <code>dia_semana</code> , <code>hora</code>	Potencia la codificación RLE de Parquet y reduce el tamaño en disco.
Índice covering por pregunta en SQLite	Cada consulta recibe únicamente el índice que necesita, en vez de construir tres y usar uno.

El artefacto Parquet de la versión XL pesa 89 MB frente a los 4,725 MB del CSV original: una reducción de 53×.

6.3. Impacto medido

Tiempo total de las tres preguntas, antes y después, en segundos.

En Pc_Jesus, versión XL, las mejoras por pregunta llegan a 110× en Pandas Q2 (57.1 s → 0.52 s).

Cuadro 9: Impacto de la optimización por versión y equipo

Versión	Equipo	Pandas	DuckDB	SQLite	ETL (s)
M	Pc_Jesus	34.5 → 0.7 (48×)	5.0 → 0.6 (8×)	108.2 → 61.7 (1.8×)	8.4
M	Pc_Martha	50.0 → 2.5 (20×)	6.7 → 2.0 (3×)	164.6 → 184.4 (0.9×)	2.9
M	Pc_Nahin	143.1 → 3.9 (36×)	15.3 → 3.8 (4×)	445.0 → 221.7 (2.0×)	8.4
L	Pc_Jesus	79.1 → 3.2 (25×)	11.5 → 1.0 (11×)	232.8 → 129.4 (1.8×)	2.6
L	Pc_Martha	106.5 → 4.6 (23×)	15.3 → 2.7 (6×)	404.0 → 304.1 (1.3×)	5.4
L	Pc_Nahin	288.1 → 6.0 (48×)	25.9 → 6.5 (4×)	776.9 → 483.8 (1.6×)	5.4
XL	Pc_Jesus	195.4 → 3.4 (57×)	27.3 → 2.7 (10×)	472.6 → 291.5 (1.6×)	6.3
XL	Pc_Martha	falló → 6.0	falló → 3.1	falló → 486.0	10.6
XL	Pc_Nahin	501.4 → 14.4 (35×)	122.3 → 14.6 (8×)	ver nota §6.5	14.8

6.4. Amortización del ETL

El ETL se paga una sola vez y el ahorro se cobra en cada corrida posterior. En la versión XL sobre Pc_Jesus, el ETL cuesta 6.3 s y ahorra 191.9 s por corrida en Pandas: **se amortiza en el 3 % de una corrida**. En DuckDB, que ya era rápido, el ahorro es de 24.6 s por corrida y la amortización toma el 26 % de una corrida. En un pipeline que se ejecuta a diario la inversión se recupera de inmediato; para un análisis único aislado, el cálculo es menos evidente, y ese contraste es parte del resultado.

6.5. Nota sobre SQLite en la versión XL de Pc_Nahin

Una lectura superficial de la gráfica sugiere que la optimización empeoró SQLite en esa máquina: 1,099.2 s en base contra 1,157.8 s optimizado. **La comparación no es válida**. En la etapa base, Q2 tardó 678.7 s y quedó marcada como EXCEDE por superar los 10 minutos, lo que hizo que el harness omitiera automáticamente Q3. Ese 1,099.2 es la suma de **dos** preguntas; el 1,157.8 es la suma de **tres**.

Comparando pregunta contra pregunta:

Cuadro 10: SQLite XL en Pc_Nahin, comparación pregunta contra pregunta

Pregunta	Base (s)	Optimizado (s)	Mejora
Q1	420.5	237.8	1.77×
Q2	678.7 (EXCEDE)	539.5	1.26×
Q3	no ejecutada	380.5	de imposible a viable

La optimización no solo ayudó: convirtió una ejecución que violaba el límite del enunciado en una que lo cumple.

6.6. Por qué SQLite gana menos que las demás

SQLite es la herramienta más castigada en la línea base y la que más gana en términos absolutos con el esquema estrecho. Pero **su ingesta sigue siendo proporcional al número de filas**: el `executemany` tiene que insertar los mismos millones de registros, solo que con campos más baratos de parsear. Por eso su mejora es grande pero no del orden de la de Pandas, y sigue sin alcanzar a DuckDB.

En Pc_Martha, versión M, la optimización incluso resultó marginalmente más lenta (164.6 s contra 184.4 s). La causa es que construir el índice covering sobre 9.6 millones de filas cuesta más de lo que ahorra el parseo del CSV estrecho en esa máquina. **Una optimización no es universalmente buena: su valor depende de dónde estaba el cuello de botella.**

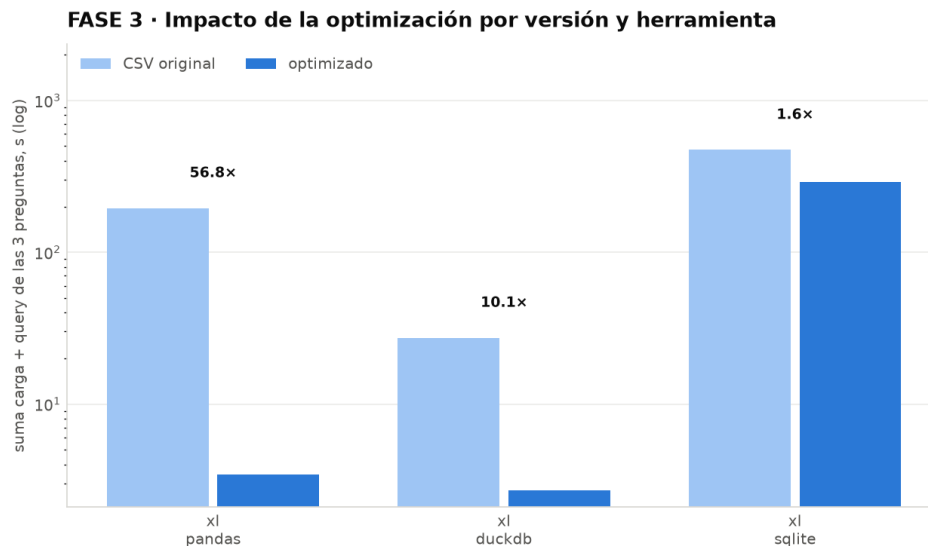


Figura 3: Impacto de la optimización por herramienta, Pc_Jesus (base vs. optimizado).

7. Comparación entre equipos

Esta es la sección que responde la pregunta central del módulo: **¿en qué punto un problema se vuelve Big Data?** La respuesta que arrojan las mediciones es que no existe un punto, sino un umbral distinto por máquina y por representación de los datos.

7.1. Dónde se rompe cada máquina

Cuadro 11: Punto de quiebre por equipo

Equipo	RAM	Punto de quiebre en línea base	Con el artefacto optimizado
Pc_Jesus	8.0 GB	Ninguno. Completó M, L y XL en las tres herramientas	Completó todo
Pc_Martha	15.7 GB	Falló en XL a los 17 minutos , antes de terminar de leer el CSV	Completó XL en 6.0 s con Pandas
Pc_Nahin	39.4 GB	SQLite cruzó los 10 minutos en XL Q2 (678.7 s) y arrastró la omisión de Q3	Completó las tres preguntas

7.2. Las tres lecturas

La máquina con menos RAM sobrevivió donde una con el doble falló. El MacBook de 8 GB completó la versión XL en las tres herramientas, mientras el equipo de 15.7 GB abortó leyendo el mismo archivo. La memoria unificada y la velocidad del SSD compensaron la diferencia nominal de RAM. Esto descarta la lectura ingenua de que más memoria significa mayor capacidad de procesamiento.

Cambiar la representación movió el umbral sin tocar el hardware. La misma Pc_Martha que no pudo leer el CSV de 4.7 GB procesó la versión XL en 6.0 segundos sobre el Parquet. El dataset no cambió, la máquina no cambió, solo cambió cómo estaban escritos los bytes. Si el umbral de Big Data se puede mover con un paso de ETL, entonces el umbral nunca fue una propiedad del dataset.

La arquitectura del motor domina sobre las especificaciones del equipo. DuckDB en la máquina de 8 GB (2.7 s en XL optimizado) superó ampliamente a SQLite en la máquina de 40 GB

(varios minutos). Estructurar bien los datos y elegir un motor analítico adecuado rinde más que agregar memoria.

7.3. Escalado

En escala logarítmica, una recta de pendiente 1 indica escalado lineal $O(n)$. Lo relevante no es la pendiente sino dónde se rompe la linealidad: cuando los datos dejan de caber cómodamente en RAM, el sistema pagina a disco y la curva se dispara. Ese quiebre es distinto en cada máquina y es, precisamente, la frontera del Big Data para ese equipo.

8. Respuestas a las tres preguntas

Las tres herramientas producen resultados idénticos, ya validados celda a celda, así que las respuestas se calculan una sola vez sobre la **versión XL: 38,215,560 viajes**, la más grande disponible.

8.1. Q1: Demanda por hora del día y día de la semana

El patrón es bimodal en días laborables y unimodal en fin de semana. La forma de las dos curvas es distinta, y esa diferencia es la respuesta.

De lunes a viernes aparecen dos picos de desplazamiento al trabajo: **08:00 h** con un promedio de 710,450 viajes y **17:00-18:00 h** con 660,092 y 606,309. Sábado y domingo tienen un único máximo aplanado a las **14:00 h**, con 439,431 viajes promedio: uso recreativo, no de transporte.

Cuadro 12: Hora pico y total por día de la semana

Día	Hora pico	Viajes en la hora pico	Total del día
Lunes	17:00	672,240	5,590,150
Martes	08:00	768,730	5,902,367
Miércoles	08:00	741,295	5,795,257
Jueves	08:00	742,080	5,795,656
Viernes	08:00	645,178	5,633,146
Sábado	14:00	442,283	4,914,592
Domingo	14:00	436,579	4,584,392

El pico laborable es 1.62 veces el pico de fin de semana. El valle nocturno está a las 03:00 h con 8,638 viajes promedio, y la franja de 00:00 a 05:00 h concentra apenas el **2.0 % del uso semanal**.

Uso operativo: las horas pico marcan cuándo la flota debe estar completa. La franja nocturna, con un 2 % del tráfico, es la ventana natural para mantenimiento y rebalanceo sin afectar el servicio.

Una advertencia de interpretación: el rango de fechas depende de la versión. Las versiones chicas cubren pocas semanas y pueden caer en período navideño, donde el patrón laborable aparece atenuado porque varios días hábiles son feriados. Comparar el mapa de calor entre versiones es en sí mismo un resultado, porque muestra que con pocos datos se puede llegar a una conclusión equivocada sobre estacionalidad.

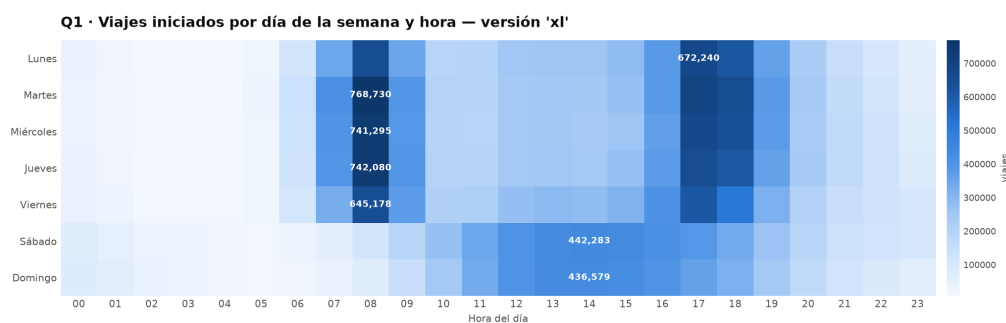


Figura 4: Mapa de calor de viajes por día de la semana y hora, versión XL.

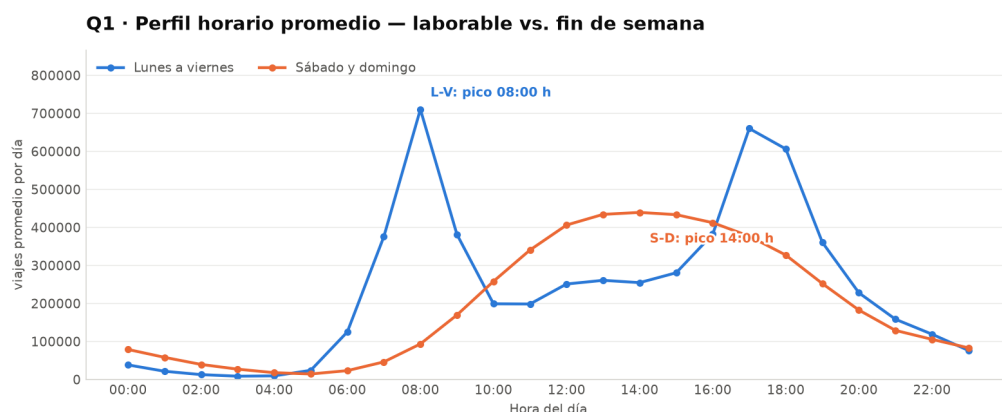


Figura 5: Perfil horario: días laborables vs. fin de semana, versión XL.

8.2. Q2: Estaciones con mayor desequilibrio

El desbalance sigue el patrón radial clásico: déficit en nodos de transporte y periferia residencial, superávit en el núcleo financiero y comercial. Con $\text{neto} = \text{llegadas} - \text{salidas}$, un valor negativo significa que la estación se vacía y hay que abastecerla; uno positivo, que se satura y hay que retirar bicicletas.

Cuadro 13: Las cinco estaciones que más se vacían

Estación	Salidas	Llegadas	Neto	Desequilibrio
Waterloo Station 2, Waterloo	125,584	87,133	-38,451	-18.1 %
Eagle Wharf Road, Hoxton	107,628	86,866	-20,762	-10.7 %
Cloudesley Road, Angel	43,919	26,874	-17,045	-24.1 %
Knightsbridge, Hyde Park	51,202	37,693	-13,509	-15.2 %
Notting Hill Gate Station, Notting Hill	89,800	77,283	-12,517	-7.5 %

Waterloo Station encabeza el déficit, lo que confirma el patrón: la gente llega en tren y toma una bicicleta hacia el centro por la mañana. Holborn, Borough y St. James's son distritos de oficinas, y ahí es donde esas bicicletas terminan.

Magnitud absoluta contra magnitud relativa. El neto ordena por volumen, pero el porcentaje revela estaciones medianas con desbalance proporcional severo. Filtrando a estaciones con más de 20,000 viajes, los casos más críticos en términos relativos son **New North Road 1, Hoxton** (-26.9 %), **Risinghill Street, Angel** (-25.2 %) y **Cloudesley Road, Angel** (-24.1 %). El barrio de Angel aparece tres veces: es una zona residencial que se vacía sistemáticamente cada mañana. Para operaciones el porcentaje suele importar más que el absoluto, porque una estación que pierde una cuarta parte de su inventario está vacía buena parte del día.

Cuadro 14: Las cinco estaciones que más se saturan

Estación	Salidas	Llegadas	Neto	Desequilibrio
Hop Exchange, The Borough	176,231	237,045	+60,814	+14.7 %
Holborn Circus, Holborn	117,818	174,057	+56,239	+19.3 %
St. James's Square, St. James's	99,768	131,601	+31,833	+13.8 %
Brushfield Street, Liverpool Street	147,822	178,519	+30,697	+9.4 %
Soho Square, Soho	123,236	146,180	+22,944	+8.5 %

Sesgo declarado: 68,213 viajes (0.18 % del total) no tienen estación de destino registrada. Cuentan como salida y no como llegada, así que empujan el neto hacia el déficit. El efecto es pequeño frente a las magnitudes reportadas, pero existe.

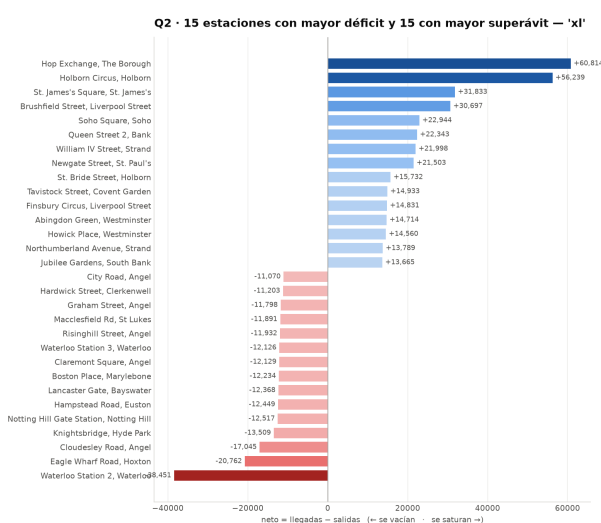


Figura 6: Estaciones con mayor déficit y superávit, versión XL.

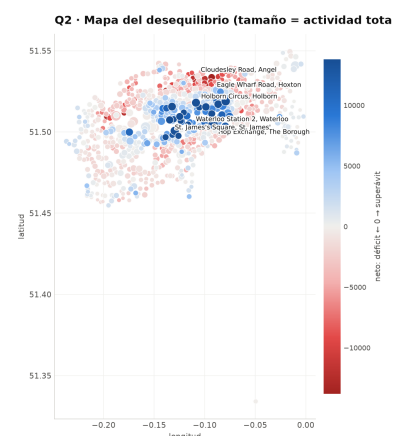


Figura 7: Mapa geográfico del desbalance por estación, versión XL.

8.3. Q3: Concentración del tráfico en rutas

Sí hay concentración, pero no es un Pareto 80/20: la forma es de cola extremadamente larga. Sobre 652,864 pares origen-destino teóricamente posibles (808 estaciones al cuadrado), solo 510,672 aparecen alguna vez. Las **1,000 rutas más transitadas concentran el 9.6 % de todos los viajes**, y las 20 primeras apenas el 1.35 %.

Cuadro 15: Rutas más transitadas, versión XL

#	Ruta	Viajes	% del total
1	Hyde Park Corner → Hyde Park Corner	72,625	0.190 %
2	Aquatic Centre → Aquatic Centre (Olympic Park)	67,661	0.177 %
3	Albert Gate → Albert Gate (Hyde Park)	42,049	0.110 %
4	Black Lion Gate → Black Lion Gate (Kensington Gardens)	40,221	0.105 %
5	Triangle Car Park → Triangle Car Park (Hyde Park)	35,652	0.093 %
10	Hyde Park Corner → Triangle Car Park	15,739	0.041 %
11	Hyde Park Corner → Albert Gate	15,405	0.040 %

Tres lecturas:

1. **Las rutas circulares dominan el top y no son viajes de transporte.** Las nueve rutas más transitadas tienen origen igual a destino, y todas están en parques: Hyde Park, Kensington

Gardens y el Queen Elizabeth Olympic Park. Son paseos recreativos, no desplazamientos. En las primeras 1,000 rutas hay 208 circulares que suman 1,026,226 viajes. Mezclarlas con los desplazamientos punto a punto distorsiona cualquier conclusión sobre movilidad, así que se marcan aparte.

2. **Los corredores estructurales aparecen después.** Las rutas no circulares del top conectan puntos dentro de los mismos parques o nodos de transporte con distritos de oficinas. Son el último tramo del trayecto al trabajo y candidatas naturales a carril exclusivo o a más capacidad de anclajes.
3. **La cola larga justifica la red.** Que ninguna ruta individual supere el 0.2 % del tráfico significa que el valor del sistema está en la cobertura, no en unos pocos corredores. Optimizar unas decenas de rutas tiene impacto local; mantener la red completa es lo que la hace útil como red.

Relación entre Q2 y Q3. Una ruta asimétrica, con mucho tráfico A→B y poco B→A, es la **causa** del desequilibrio que Q2 mide como **efecto**. Cruzar ambas identifica qué corredor concreto genera cada estación deficitaria, que es la información que necesita el equipo de rebalanceo. Y cruzar Q2 con Q1 daría la ventana horaria en que conviene intervenir, porque una estación puede vaciarse por la mañana y llenarse por la tarde.

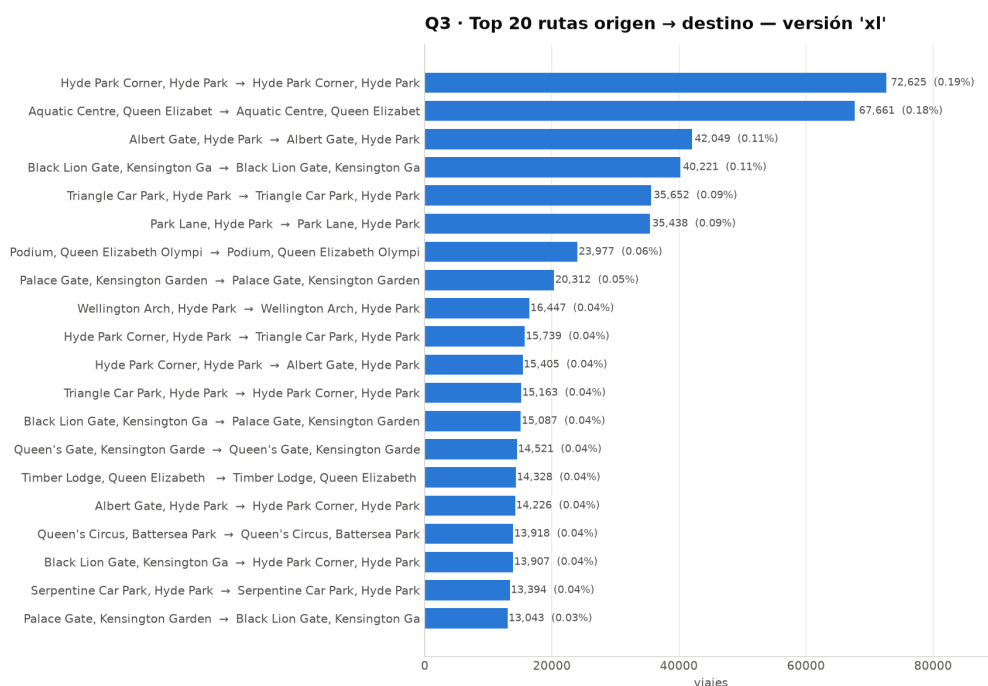


Figura 8: Top 20 rutas más transitadas, versión XL.

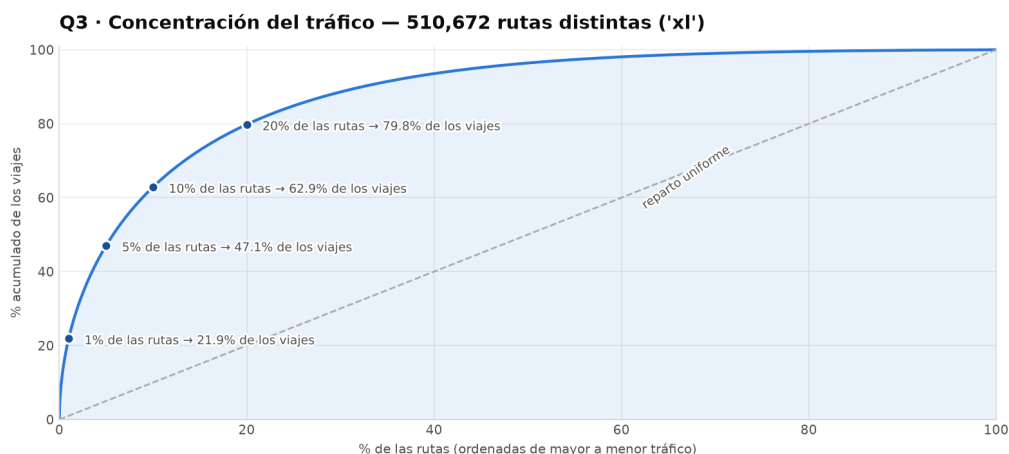


Figura 9: Curva de concentración (Pareto) del tráfico por ruta, versión XL.

9. Diseño del pipeline ETL

El enunciado pide únicamente el diagrama, sin implementación: lo que sigue es la propuesta, sobre Databricks + Azure. El ETL medido en la Fase 3 ya es, de hecho, un pipeline de una sola capa. Formalizarlo en capas lo vuelve reutilizable y explica las decisiones de diseño. Se asume un escenario de ingesta periódica: el operador del sistema de bicicletas entrega un archivo mensual con los viajes nuevos, de un tamaño similar a la versión S del dataset (0.488 GB, 3.97 millones de filas). Con esa cadencia, el archivo acumulado en la capa bronce alcanza el tamaño de la versión XL (4.725 GB) alrededor del mes 10, momento que se usa como estado estable para el análisis de costos de la Sección 10.

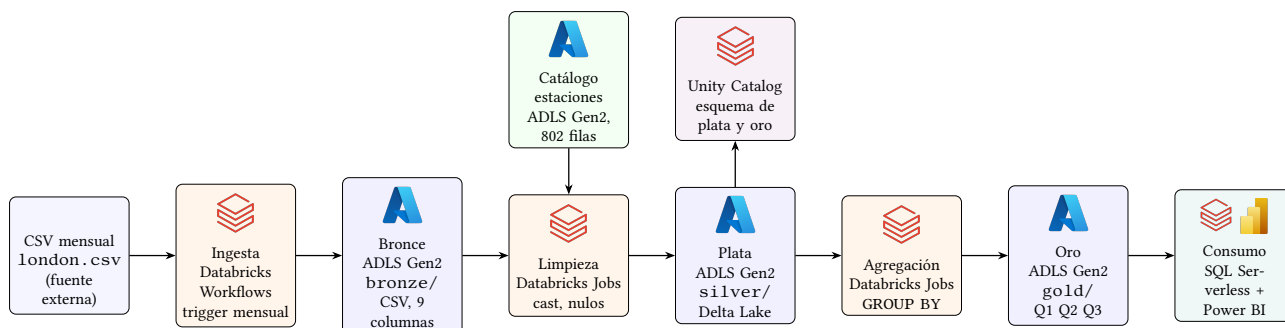


Figura 10: Pipeline ETL propuesto sobre Databricks + Azure, capas medallón (bronce/plata/oro) en Delta Lake sobre Azure Data Lake Storage Gen2, con Databricks Jobs como motor de transformación, Unity Catalog como catálogo de esquema y Databricks SQL Serverless + Power BI para el consumo. Íconos oficiales de Microsoft Azure, Databricks y Power BI.

9.1. Herramientas y servicios por etapa

La capa de limpieza es donde se resuelven los tres problemas de calidad detectados en la exploración: el casteo de `end_station_id` a entero, el tratamiento de los nulos de destino y el marcado de duraciones anómalas. La capa plata materializa la proyección de cuatro columnas con tipos reducidos que produjo las mejoras de la Fase 3.

Pendiente de decidir por el equipo:

- Confirmar si el enunciado pide capas medallón u otra nomenclatura

Cuadro 16: Servicios de Databricks sobre Azure asignados a cada etapa del pipeline

Etapa	Servicio	Por qué esa herramienta
Orquestación	Databricks Workflows (Jobs)	Programa el job mensual sobre el mismo clúster que ejecuta el ETL; sin servicio de orquestación aparte encendido de forma permanente.
Ingesta / Bronce	ADLS Gen2 (contenedor bronze/)	Almacenamiento jerárquico, barato y duradero para el CSV tal cual llega. También aloja el catálogo de estaciones.
Limpieza y transformación	Databricks Jobs (notebook PySpark)	Motor Spark administrado: resuelve casteo, nulos y proyección a 4 columnas; el mismo trabajo de Fase 3, escalado a un job programado.
Plata	ADLS Gen2 (silver/), Delta Lake	Sigue la decisión de Fase 3 (53× de reducción frente al CSV); Delta añade versionado y transacciones ACID sobre ese mismo Parquet.
Catálogo de esquema	Unity Catalog	Registra el esquema de plata y oro con control de acceso centralizado, sin definir tablas a mano.
Agregación / Oro	Databricks Jobs (mismo notebook)	Calcula Q1, Q2 y Q3 y las escribe como Delta pequeño en gold/.
Consumo	Databricks SQL (Serverless) + Power BI	El warehouse arranca en segundos, se apaga solo tras inactividad y cobra por segundo activo; evita un clúster SQL encendido para tan pocas consultas.

- Decidir si el catálogo de estaciones se une en plata o en oro
- Especificar la estrategia de particionado en ADLS Gen2, si aplica (por ejemplo, por año-mes de `start_rental_date_time`)

10. Análisis de costos

El análisis cubre dos cosas distintas: el costo de operar el pipeline ETL propuesto en la Sección 9 sobre Databricks + Azure, y el costo ya medido de la optimización de la Fase 3. Los precios son los públicos y actuales de las calculadoras oficiales de [Azure Databricks](#) y [Azure Data Lake Storage Gen2](#) para la región `us-east` (Este de EE. UU.), consultados en septiembre de 2026; no se espera exactitud, sino que el desglose sea razonable, esté sustentado y use tarifas reales en vez de estimaciones inferidas.

Supuestos del escenario:

- Proveedor: Databricks sobre Microsoft Azure, tier Premium, región East US, precios *pay-as-you-go* sin descuentos por compromiso.
- Frecuencia: batch mensual, archivo del tamaño de la versión S (0.488 GB) por corrida; estado estable en el mes 10, cuando bronce alcanza el tamaño de la versión XL (4.725 GB).
- La capa plata mantiene la razón de reducción de la Fase 3 (53× frente al CSV): el Delta acumulado en el mes 10 pesa ~89 MB.
- ETL en un nodo `Standard_DS3_v2` (4 vCPU, 14 GB RAM, 0.75 DBU), *Jobs Compute*, 10 minutos por corrida (arranque de clúster incluido).
- Consumo contra un *warehouse* Databricks SQL Serverless 2X-Small (4 DBU), que se apaga solo tras inactividad; se asumen 5 minutos de uso mensual.

Cuadro 17: Desglose de costo mensual del pipeline en Databricks + Azure, mes 10 (estado estable)

Componente	Cálculo	Costo/mes	Fuente
Almacenamiento ADLS Gen2 (bronce+plata+oro)	$(4.725 + 0.089 + 0.005) \text{ GB} \times \$0.0208/\text{GB}$ (Hot, LRS)	\$0.100	[1]
Operaciones ADLS Gen2 (escritura/lectura mensuales)	$\sim 50 \text{ escrituras} \times \$0.0625/10\text{k} + \sim 5,000 \text{ lecturas} \times \$0.004/10\text{k}$	\$0.002	[1]
Cómputo ETL (Databricks Jobs Compute, DS3 v2, 1 corrida/mes)	$\$0.518/\text{hora}$ (VM + 0.75 DBU) $\times 10 \text{ min}$	\$0.086	[2]
Orquestación (Databricks Workflows)	Incluida en el cómputo del job; no hay cargo de orquestador aparte	\$0.000	[2]
Catálogo de esquema (Unity Catalog)	Incluido en el workspace Premium a este volumen de tablas	\$0.000	[2]
Consulta (Databricks SQL Serverless, 2X-Small)	$\$2.80/\text{hora} \times 5 \text{ min/mes de uso}$	\$0.233	[2]
Transferencia de salida	Muy por debajo de los 100 GB/mes gratuitos de Azure	\$0.000	[3]
Total estimado		\$0.42/mes	
[1] azure.microsoft.com/pricing/details/storage/data-lake [2] azure.microsoft.com/pricing/details/databricks [3] azure.microsoft.com/pricing/details/bandwidth			

10.1. Costo del pipeline en la nube (mensual, estado estable)

El total sigue siendo minúsculo ($\sim \$5/\text{año}$) porque todo el histórico del dataset cabe en menos de 5 GB. Esa es en sí misma una conclusión relevante: **a esta escala, el costo de nube no es un problema**; lo que domina el presupuesto es la decisión de diseño (Delta vs. CSV, cuánto tiempo queda encendido el warehouse), no el gasto en infraestructura. Esto es consistente con el hallazgo de la Fase 3: la representación de los datos importa más que la cantidad de recursos que se les asigna.

Databricks cuesta más que un equivalente sobre servicios nativos de nube a este volumen, y la causa es identificable. Un diseño equivalente sobre S3 + Glue + Athena rondaría $\$0.27/\text{mes}$; aquí sale $\$0.42/\text{mes}$. El almacenamiento es casi igual (ambos $\sim \$0.10$); la diferencia está en el consumo: un motor facturado por *terabyte escaneado* (Athena) es más barato a este volumen que uno facturado por *tiempo encendido* (Databricks SQL Serverless, con tarifa mínima de $\$2.80/\text{hora}$). La brecha se cierra o se invierte en cuanto crece el volumen de consultas concurrentes, el escenario para el que Databricks SQL está diseñado.

Sensibilidad a la escala. Si el sistema creciera 200 veces ($\sim 1 \text{ TB}$ acumulado), el almacenamiento en bronce pasaría a $\approx \$21.3/\text{mes}$ (sigue en el primer escalón de tarifa) y el cómputo ETL a $\sim \$0.86/\text{mes}$. El costo dejaría de ser trivial, pero seguiría siendo pequeño frente a SQLite sin optimizar, como se documentó en la Fase 3.

10.2. Costo ya medido: cómputo local y tiempo de persona

Costo en almacenamiento (medido). El CSV original de la versión XL pesa 4,725 MB. El Parquet optimizado pesa 89 MB y el CSV estrecho para SQLite pesa 442 MB. La reducción es de $53\times$ y $10.7\times$ respectivamente.

Costo en cómputo local (medido). El ETL de la Fase 3 cuesta entre 2.6 s y 14.8 s según la

máquina y la versión. El ahorro por corrida completa del análisis va de 24.6 s (DuckDB, XL) a 191.9 s (Pandas, XL). La amortización ocurre en una fracción de la primera corrida.

Costo en tiempo de persona. Pc_Martha perdió 17 minutos en una ejecución que terminó fallando. Ese tipo de costo no aparece en ninguna tabla de benchmark ni en la tabla de costos de nube, pero es real y recurrente cada vez que alguien reproduce el análisis sin el artefacto optimizado.

Pendiente de decidir por el equipo:

- Validar los supuestos de frecuencia y tamaño de la carga incremental contra el enunciado, si este especifica algo distinto
- Decidir si conviene pasar el warehouse de Databricks SQL a capacidad reservada si el volumen de consultas crece

11. Problemas encontrados

Cuadro 18: Problemas encontrados durante el desarrollo

#	Problema	Cómo se resolvió
1	Las tres herramientas numeran los días de la semana distinto. Pandas <code>dt.dayofweek</code> arranca en lunes = 0; DuckDB <code>dayofweek()</code> y SQLite <code>strftime('%w')</code> arrancan en domingo = 0. Los resultados no coincidían aunque el total sí.	Se unificó todo a lunes = 0 con <code>isodow()-1</code> en DuckDB y <code>(strftime('%w')+6)%7</code> en SQLite. La validación cruzada se agregó justamente para cazar este tipo de error.
2	<code>end_station_id</code> se infiere como DOUBLE porque tiene nulos, mientras <code>start_station_id</code> es entero. Al agrupar por el par, los identificadores no cruzaban: 529.0 contra 529.	<code>CAST(... AS BIGINT)</code> y <code>.astype("int64")</code> explícitos en las tres herramientas antes de agrupar.
3	FULL OUTER JOIN no existe en SQLite anterior a la versión 3.39. La consulta de Q2, escrita originalmente para DuckDB, fallaba con error de sintaxis en instalaciones de SQLite más antiguas durante el desarrollo.	Se reescribió con un CTE <code>ids</code> (unión de los identificadores de origen y destino) más dos LEFT JOIN, y se usó la misma forma en DuckDB para que las consultas fueran estructuralmente comparables.
4	La ingesta a SQLite era lentísima con el journal y el fsync por defecto.	<code>PRAGMA journal_mode=OFF, synchronous=OFF, cache_size=-262144</code> e inserción por lotes de 200,000 filas con <code>executemany</code> , en vez de un INSERT por fila.
5	SQLite no conserva el CSV cargado entre consultas , a diferencia de Pandas y DuckDB.	Se adoptó el protocolo de ejecución independiente y se aplicó a las tres herramientas, para que la comparación fuera simétrica.
6	El import de pyarrow inflaba el tiempo de carga del Parquet. Pandas lo carga de forma perezosa en el primer <code>read_parquet()</code> , y ese import en frío se contabilizaba como tiempo de lectura de datos.	Se movió el import fuera de todo bloque cronometrado.
7	Los artefactos pesados en carpetas sincronizadas distorsionaban los tiempos. La sincronización en segundo plano compite por I/O.	Los <code>.db</code> y <code>.parquet</code> se escriben en disco local, no en la carpeta de nube.
8	Pc_Martha (15.7 GB) falló en XL tras 17 minutos sin llegar a completar la lectura del CSV.	Se registró como resultado, no como error a corregir: es el dato que define el umbral de Big Data para ese equipo. Se corrió una segunda pasada sin XL para obtener la serie completa de las demás versiones.
9	La gráfica de impacto suma tiempos sin verificar que ambas etapas tengan las mismas preguntas completadas. Cuando una etapa tiene ejecuciones omitidas, la barra de la línea base queda artificialmente corta y sugiere que la optimización empeoró el rendimiento.	Se detectó en el caso de SQLite en XL sobre Pc_Nahin y se corrigió comparando pregunta contra pregunta. La comparación válida está en la Fase 3 (§6.5).

12. Limitaciones metodológicas

Tres advertencias que condicionan cómo deben leerse los números de este informe.

El pico de RAM no es comparable entre macOS y Windows. Pandas en la versión XL reporta 4,522 MB en Pc_Jesus y 12,096 MB en Pc_Nahin, para el mismo código y los mismos datos. La diferencia es demasiado grande para ser real: macOS comprime páginas de memoria y el valor de RSS que reporta `psutil` no las contabiliza. Los números de RAM son válidos para comparar herramientas **dentro** de una misma máquina, no entre sistemas operativos distintos.

El equipo de mayores especificaciones fue el más lento. Pc_Nahin, con Core Ultra 9 y 40 GB, tarda de 2 a 4 veces más que Pc_Jesus, de 8 GB, en todas las herramientas, incluida la carga pura de CSV con DuckDB (36 s contra 8 s en XL). Esa es una brecha de entrada/salida, no de CPU ni de memoria. Las causas plausibles son que el archivo estuviera en una carpeta sincronizada con la nube, que el antivirus escaneara un archivo de 4.7 GB en cada lectura, o que las tres versiones se corrieran en una sola sesión sin liberar memoria entre ellas. **No se verificó cuál de las tres aplica**, así que los tiempos absolutos de ese equipo deben tomarse como cota superior.

Las versiones de librerías difieren entre equipos. Python va de 3.12.10 a 3.14.3 y pandas de 3.0.1 a 3.0.5. Es improbable que explique diferencias de orden de magnitud, pero introduce ruido en las comparaciones finas entre máquinas.

Las conclusiones de este informe se apoyan en diferencias de órdenes de magnitud y en fallos categóricos, no en márgenes estrechos, precisamente porque estas tres limitaciones hacen que los márgenes estrechos no sean defendibles.

13. Conclusiones

Big Data no es un tamaño, es una relación. El mismo archivo de 38.2 millones de filas fue imposible en una máquina de 15.7 GB de RAM y manejable en una de 8 GB. Ninguna cifra absoluta de filas o gigabytes marca la frontera; la marca la interacción entre el volumen, la forma en que están escritos los datos y el equipo que los procesa.

La representación es la palanca más barata. El paso de CSV a Parquet con proyección y tipos reducidos costó entre 2.6 y 14.8 segundos por versión y multiplicó el rendimiento por factores de hasta 110×. Más importante: movió el umbral de viabilidad. Un equipo que no podía procesar la versión completa pasó a hacerlo en 6 segundos, sin agregar hardware.

La arquitectura del motor importa más que sus especificaciones. DuckDB, columnar y vectorizado, resolvió en 2.7 segundos sobre 8 GB de RAM lo que SQLite, orientado a filas, no terminaba en 10 minutos sobre 40 GB. Los índices y los PRAGMA mejoran a SQLite, pero no cambian su modelo de almacenamiento: un B-tree no convierte un motor OLTP en columnar.

La carga, no la consulta, es el cuello de botella. Entre el 57 % y el 97 % del tiempo total se va en leer el archivo. Cualquier esfuerzo de optimización que ignore esto ataca el problema equivocado.

Sobre el sistema de bicicletas. El uso es bimodal en días laborables, con picos a las 08:00 y las 17:00-18:00, y unimodal y recreativo en fin de semana, con máximo a las 14:00. El desbalance logístico sigue un patrón radial: las estaciones de tren y las zonas residenciales se vacían, los distritos de oficinas se saturan. Y el tráfico está concentrado pero no dominado: las 1,000 rutas más usadas suman el 9.6 % de los viajes, y las nueve primeras son circulares dentro de parques, es decir, paseos y no desplazamientos.